NOTE: Apart from this page, we have clicked pictures of only those pages where solutions had to be written. Also, graphs are in later section of the PDF, as they were made on MATLAB

STAGE 2 – PRACTICAL ENGINEERING CHALLENGE

TEAM ID:	TEAM NAME:
LRC-26-0099	CADENCE

Problem Statement: Industrial Manufacturing Process Automations

Industrial manufacturing depends on highly controlled liquid-handling systems to deliver process fluids reliably from a source or bulk supply to downstream process equipment. Depending on the application, these fluids may include **ultrapure fluids, process chemicals, solvents, developers, or CMP-related fluids**. Industry standards explicitly address liquid-chemical distribution, high-purity piping, bulk chemical distribution, system qualification, and UPW system operation and monitoring.

In such systems, **flow is a fundamental process variable**. An incorrect flow rate can indicate or cause a restriction, incorrect valve state, inadequate pumping performance, abnormal process demand, or another hydraulic deviation. The consequence may be unstable process conditions, incorrect chemical delivery, equipment malfunction, or loss of process performance.

This challenge therefore focuses on a fundamental industrial facilities and process-engineering task: **To establish a defined process-fluid flow, measure it reliably, detect a deviation, diagnose its cause, and restore the process to the required operating condition.**

Engineering Challenge

As you are part of a multidisciplinary engineering team responsible for designing, building and validating a small-scale automated liquid-flow system represented of industrial process handling system.

CASE 1 – CONTROL + MEASUREMNT

Problem: Develop a system that manages the changing level of fluid in the sump automatically (when there is a sudden external fluid charging to the sump) and while accurately measuring and totalizing the fluid discharged from the sump using the flow transmitter.

CASE 2 – FLUSHING +SHUTDOWN

Problem: Develop an alternative operating configuration that allows flush water to pass through the system while all three pumps operate simultaneously, and then safely transition the system to an OFF condition when the flush-water supply has been exhausted.

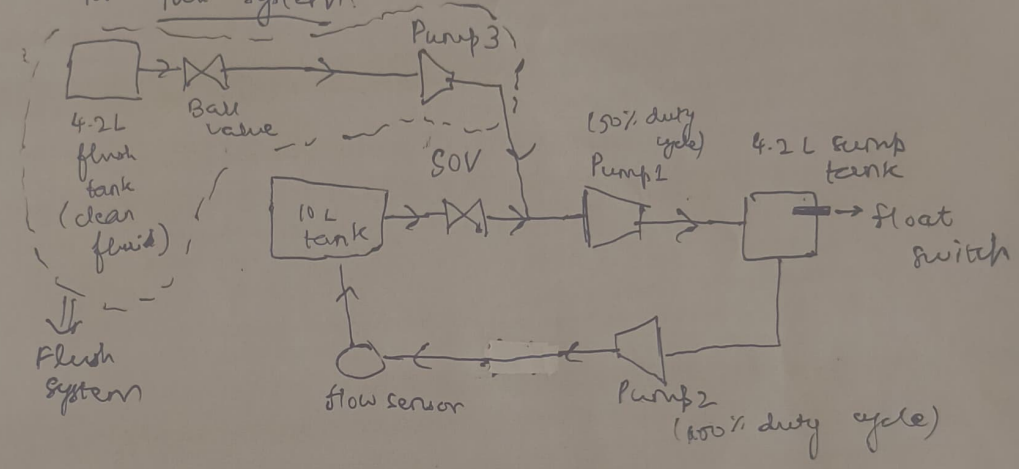
Given Process Parameters

Pipe diameter =	8mm
Fluid density (ρ) =	1000kg/m^3
Dynamic Viscosity (μ) =	1x10^{-3}Pa.s
Required Flow range =	1-6 L/min
Nominal target flow for the live challenge =	3L/min

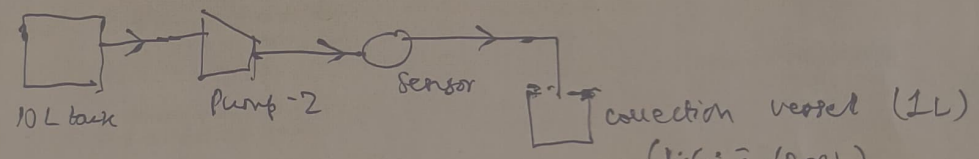
Question 1-L Solution

P&ID SKETCH / WORKING SPACE

P&ID for full system:



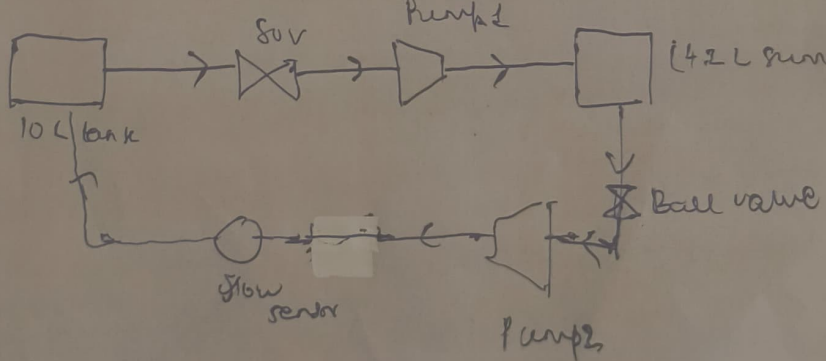
Calibration setup: (Q2.2 & Q 2.3)



(Flush system disabled)

Flow rate trend observ setup ⊕ Value response set-up

value response (Q2.1 and Q1.2) for observing flow rate



~~[illegible]~~

* At higher flowrate, flow is more aggressive (the water jet coming from the pipe splashes against the vessel wall for higher flowrates)

Q1.2 After setting-up the system, now operate it at approximately at 1L/min, 3L/min & 6L/min. Compare the theoretical velocity with the actual operating behaviour.

WORKING / COMPARISON

Theoretical velocity = flow rate (L/min) $\times \frac{10^{-3}}{60} \times \frac{4}{\pi d^2}$

$= \text{flow rate (L/min)} \times \frac{1}{6\times10^{4}} \times \frac{4}{\pi \times 64\times10^{-6}}$

$= [0.33 \times \text{flow rate (L/min)}]$ in m/s

⇒ Theoretical velocities: 0.33 m/s, 0.99 m/s and 1.98 m/s respectively.

Experimental observations:

* The max flow rate for an individual pump that was observed during calibration was around 2.3 L/min.
* However, in the system, while circulating water in the drop (b/w 10L & 4.2L sump tank, flush opp), the max. flow rate observed was around 1.5 L/min. This was counter-intuitive, as the both pumps were at 100% duty cycle. Thus upon closely observing the flow through the transparent pipe, we saw the fluid become frothy & whitish, very likely due to flow separation (due to momentum loss). This was due to pressure losses, because of bends in pipes, sudden turns, long pipe length.
⇒ Hence, we decided to reduce pipe lengths in those segments.
* After this, the flow rate observed (both pumps @ max duty cycle) was around 2.6 L/min (73% ↑ in flow rate).
* Depending on initial water levels in 10L & 4.2L tank, the flow rate changed for same operating condition of pumps due to the contribution of the hydrostatic pressure (ρgh).
* Note: $Re = \frac{\rho U D}{\mu}$ = [illegible] and $U = Q/A$

$= \frac{10^{3}}{10^{-3}} \times \frac{2.6\times10^{-3}}{60} \times \frac{4}{\pi D^2} \times \emptyset = \frac{2.6\times4\times10^{3}}{60\pi D} = \frac{10.4\times10^{3}\times10^{3}}{60\times\pi\times8} = 6846.7142 > 3000$

(very turbulent flow)

→ * Also, when duty cycle of both pumps = 50%, flow rate ↓ as expected (≈ 1.2 L/min)
* When default operating conditions (PL → 50%, PL → 100%) ≈ 2.4 L/min

TASK 2: FLOW RESPONSE TO VALVE POSITION

Q 2.1 Set the ball valve to approximately 25%,50%,75% & 100%. At each position, allow the flow to stabilize and record the flow rate. Create the following graph of valve position (%) vs Flow Rate (L/min)

VALVE POSITION	FLOW RATE
25%	0.525
50%	1.231
70%	1.357
100%	1.398

(L/min)

GRAPH

(In the MATLAB code PDF that has been attached) It has the graph.

Q2.2 Determine the experimental calibration equation: $Q = af + b$, where Q = flow rate in L/min, f = pulse frequency in Hz, a = experimental calibration coefficient and b = offset.

Report:

R^2 =	0.9973599
a =	~~0.20~~ 0.0207807
b =	0.1091725
Maximum calibration error =	0.03728 L/min (absolute value)
Average calibration error =	(Mean absolute error) 0.02108 L/min

Q2.3 FT-01 produces pluses corresponding to water flow. Run the system at least for 5 different flow conditions within the available operating range (Hint: use formulas f=N/t and Q=V/t). For each condition, record the following:

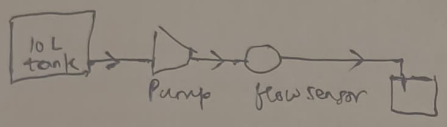
Trial	Pulses	Time(s)	Frequency (Hz)	Collected Volume (L)	Actual Flow (L/min)
1	2522	24.511	102.89	0.920	2.25
2	2598	27.727	93.7	0.940	2.63
3	2489	29.757	83.64	0.920	1.86
4	2394	40.332	59.36	0.930	1.38
5	2341	56.752	41.25	0.890	0.94

Graph is the MATLAB code PDF that has been attached. MATLAB code also has the relevant calculations done for each data point.

CALCULATIONS / OBSERVATIONS

CALIBRATION SET-UP:

Flow rate was changed by changing the duty cycle of PWM input to pump.



collection vessel with pre-marked levels for measuring volume. capacity = 2L and least count = 10mL.

* CIRCUIT & MCU communication:
Motor driver was powered using 12VDC from SMPS and there was no power input to PCB. ESP was connected to laptop using micro-USB and was powered by that only. The float sensor pulses being sent to ESP pin were detected using INTERRUPT (ISR) in the code, and communication with laptop is through UART (micro-USB connection).

* CALCULATIONS (1st reading)

$t_1 = 30.411808$; $t_2 = 31.936319$, $\Delta t = 24.5110$ | * in min:sec.msec.microsec. for $i = 1, 2$

No. of pulses = 2522

Frequency = No. of pulses / Δt = 2522/24.511 = 102.89 Hz

$V_{collected}$ = 920mL ⇒ Flow rate = $V_{collected}/\Delta t$ = 920/24.511 = 2.25 L/min

(refer MATLAB code for Gaussian regression in the end)

Process Used –
The code published the timestamped pulse count of flowmeter on the serial output. The data was mostly linear in steady state (in the plot) and had quickly increasing slope at start of pump (sometimes weren't visible on graph) and a bit slower decreasing slope in the end. The fluid was collected in a measuring jar. We observed the last and second last timestamp, pulsecount only differed by 1. Since operating range of flow meter is > 0.3 L/min it can be assumed that this last pulse was not counted properly. Soo, we discarded the last timestamps. Q and f are linearly related ⇒ Q_{avg} and f_{avg} also linearly related. So we can include the non linear regions as well.

Results and Discussion ($\alpha = 0.020781$ +- 0.00972)
Regression on data gave $R^2 > 0.997$. Ideally $k = 0$, i.e. @ $Q = 0$, $f = 0$. But non zero k can be attributed to pump initially running outside (<0.3L/min) range of flow meter among other reasons of like miscounting. Therefore, we made sure at $f = 0$, $Q = 0$ is outputted by the code for setup. For non zero f, where we operate at high flow rates, we use the calibration equation.

Team ID: LRC260099 Team Name: Cadence

TASK 2: HYDRAULIC SYSTEM THEORY AND FAULT ANALYSIS

Q3.1 The effective liquid volume inside a section of tubing is 0.5L, while the operating flow rate is 2.5L/min. The hydraulic residence time is given by $T_r = Q/V$. Calculate the residence time in seconds, also answer the following questions:

1. What does residence time physically represent in this system?
2. What happens to residence time if the tubing volume is doubled while the flow rate remains constant?
3. What happens to residence time if the flow rate is doubled while the tubing volume remains constant?
4. Why is residence time important when interpreting a sudden change in measured flow?

ANSWER $T_r = \frac{V}{Q}$, given $V = 0.5L$ and $Q = 2.5 L/min$

$\therefore T_r = \frac{0.5}{2.5} = 0.2 mins = 12 seconds.$

1)→ residence time represents the time taken by the fluid to move through a section of tubing.

2)→ if $V \rightarrow 2V$ and Q is kept constant, then $T_r = \frac{V}{Q}$ residence time will be doubled.

3)→ if $Q \rightarrow 2Q$, and V is kept constant, then residence time will be halved.

4)→ A change at the pump or valve takes about T_r time to reach FT-01, which causes the sensor to lag. If you act before T_r, then you will misread the fault's timing, mistake a transient for a steady state, or overcorrect
Large tubing volume means a longer lag and slower fault detection

Team ID: LRC 26-0099 Team Name: Coplener

Q3.2 The following observation are obtained from the same system:

CONDITION	PUMP	VALVE	FT-01 PULSE SIGNAL	MEASURED FLOW
A	ON	Open	Normal	Normal
B	ON	Partially closed	Reduced	Reduced
C	ON	Open	Zero	Zero
D	OFF	Open	Zero	Zero
E	ON	Open	Intermittent	Fluctuating

For each of the condition, identify the most likely physical or instrumentation condition. Then answer: Which condition is most difficult to distinguish using FT-01 alone, and what additional information would you require to distinguish the fault? (Hint: Teams must consider possibilities such as: Valve restriction, pump failure, hydraulic blockage, sensor/signal failure, unstable flow etc)

condition — Most likely cond. — Reasoning

>> A ⟶ normal operation ⟶ pump on, valve open, pulse and flow agree.

>> B ⟶ valve restriction → partially closed valve adds resistance, so real flow drops and the pulse rate falls with it. sensor is reading correctly.

>> C ⟶ hydraulic blockage, pump failure, or sensor/signal failure → pump is on, valve is ~~open~~ so zero flow is unexpected. A real zero and a false zero flow ~~are~~ look identical

>> D ⟶ pump off
↳ reasoning → no driving force, so zero pulse and zero flow are correct

>> E ⟶ unstable flow, or an intermittent signal fault.
↳ reasing → pump on and valve open but pulses and flow are erratic,

Hardest to distinguish using FT-01 alone: Condition C
FT-01 reads zero ~~both~~ when fluid is truly not moving and when fluid is moving but sensor or signal has failed. The pulse signal cannot tell them apart.

(remaining answer on next page)

Additional information required.

>> pump discharge and suction pressure (and differential pressure across strainer or line sections)

>> motor current or power draw

>> independent flow confirmation (second flowmeter, clamp on ultrasonic, tank level change)

>> sensor checks (supply voltage, raw signal, wiring continuity)

How these seperate the faults:

>> high discharge pressure with normal motor current → hydraulic blockage

>> low pressure with low motor current → pump failure or loss of prime.

>> normal pressure and current, with independent flow present ∴ FT-01 or signal failure.

Conclusion

FT-01 reports a symptom, not a cause. Pressure, motor current and an independent flow reference are needed to seperate valve restriction, pump failure, blockage and sensor failure.

Table of Contents

..... 1
Q2.3: Experimental data 1
Q2.2: Linear regression 1
Calculate R-squared 1
Calculate calibration errors 1
Display calibration results 2
Display trial-wise results 2
Plot experimental data and regression line 3

```
clc;
clear;
close all;
```

Q2.3: Experimental data

Pulse frequency (Hz)

```
f = [102.89; 59.36; 83.64; 41.25; 93.70];

% Actual flow rate (L/min)
Q_actual = [2.25; 1.38; 1.86; 0.94; 2.03];
```

Q2.2: Linear regression

Calibration equation: Q = a*f + b

```
p = polyfit(f, Q_actual, 1);

a = p(1);                   % Slope
b = p(2);                   % Offset

% Predicted flow rates
Q_predicted = polyval(p, f);
```

Calculate R-squared

```
SSE = sum((Q_actual - Q_predicted).^2);
SST = sum((Q_actual - mean(Q_actual)).^2);

R2 = 1 - SSE/SST;
```

Calculate calibration errors

```
residual = Q_predicted - Q_actual;
absolute_error = abs(residual);
```

```
max_error = max(absolute_error);
MAE = mean(absolute_error);
```

Display calibration results

```
fprintf('\n========== CALIBRATION RESULTS ==========\n');
fprintf('Calibration equation: Q = a*f + b\n\n');

fprintf('Slope, a                  = %.7f\n', a);
fprintf('Offset, b                 = %.7f\n', b);
fprintf('R-squared                 = %.7f\n', R2);
fprintf('Maximum absolute error    = %.5f L/min\n', max_error);
fprintf('Mean absolute error (MAE) = %.5f L/min\n', MAE);
```

```
========== CALIBRATION RESULTS ==========
Calibration equation: Q = a*f + b

Slope, a                  = 0.0207807
Offset, b                 = 0.1091725
R-squared                 = 0.9973599
Maximum absolute error    = 0.03728 L/min
Mean absolute error (MAE) = 0.02108 L/min
```

Display trial-wise results

```
Results = table(f, Q_actual, Q_predicted, ...
    residual, absolute_error, ...
    'VariableNames', {'Frequency_Hz', ...
    'ActualFlow_Lmin', 'PredictedFlow_Lmin', ...
    'Residual_Lmin', 'AbsoluteError_Lmin'});

fprintf('\n========== TRIAL-WISE RESULTS ==========\n');
disp(Results);
```

```
========== TRIAL-WISE RESULTS ==========
    Frequency_Hz    ActualFlow_Lmin    PredictedFlow_Lmin
Residual_Lmin    AbsoluteError_Lmin
    ____________    _________________    __________________
_____________    __________________

       102.89            2.25                2.2473
-0.002697           0.002697
        59.36            1.38                1.3427
-0.037283           0.037283
        83.64            1.86                1.8473
-0.012726           0.012726
        41.25            0.94               0.96638
0.026378           0.026378
         93.7            2.03                2.0563
```

```
0.026328           0.026328
```

Plot experimental data and regression line

```
f_fit = linspace(min(f), max(f), 200);
Q_fit = polyval(p, f_fit);

figure('Color', 'w');

scatter(f, Q_actual, 80, 'b', 'filled');
hold on;

plot(f_fit, Q_fit, 'r-', 'LineWidth', 2);

grid on;
box on;

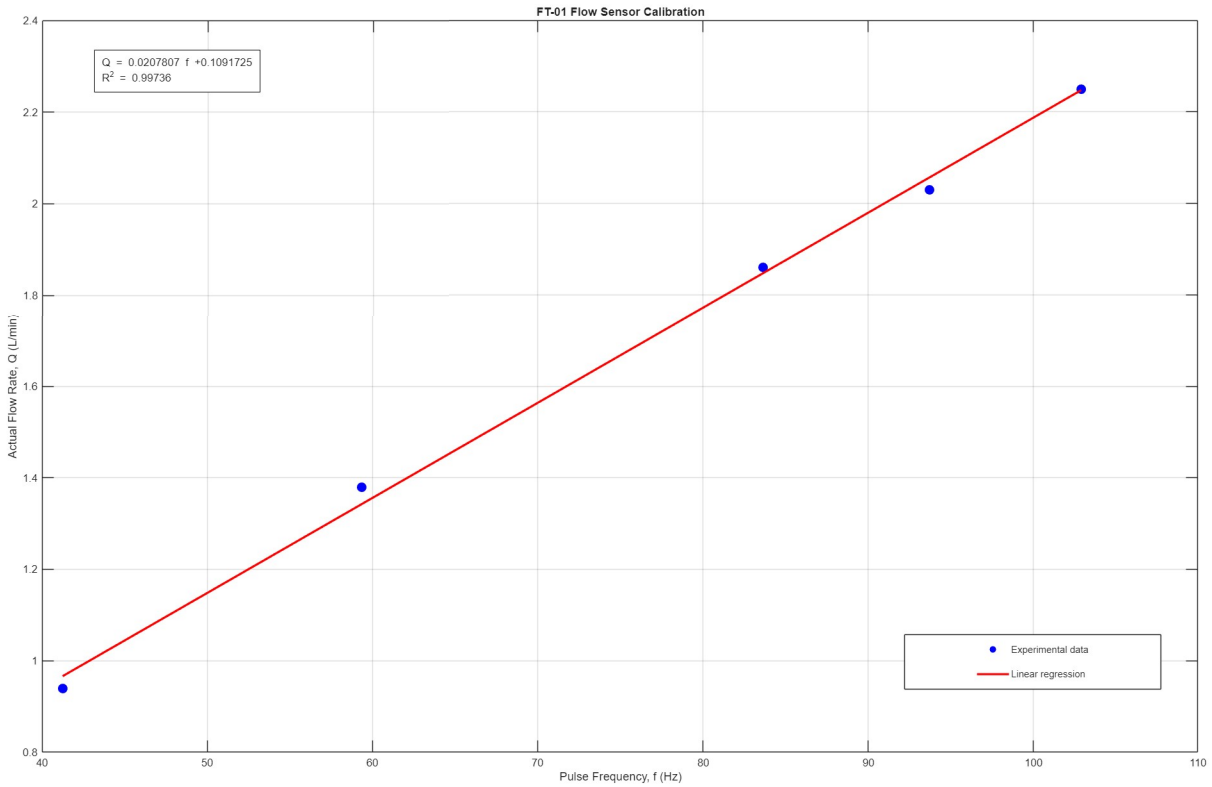
xlabel('Pulse Frequency, f (Hz)');
ylabel('Actual Flow Rate, Q (L/min)');
title('FT-01 Flow Sensor Calibration');

legend('Experimental data', 'Linear regression', ...
    'Location', 'northwest');

% Add equation and R-squared to the graph
equation_text = sprintf( ...
    'Q = %.7f f %+.7f\nR^2 = %.5f', a, b, R2);

text(0.05, 0.95, equation_text, ...
    'Units', 'normalized', ...
    'VerticalAlignment', 'top', ...
    'BackgroundColor', 'white', ...
    'EdgeColor', 'black', ...
    'Margin', 6);

hold off;
```


FT-01 Flow Sensor Calibration
Q = 0.0207807 f +0.1091725
R2 = 0.99736
Actual Flow Rate, Q (L/min)
Pulse Frequency, f (Hz)
Experimental data
Linear regression
2.4
2.2
2
1.8
1.6
1.4
1.2
1
0.8
40
50
60
70
80
90
100
110

```
% Q2.1: Valve Opening vs Flow Rate

clc;
clear;
close all;

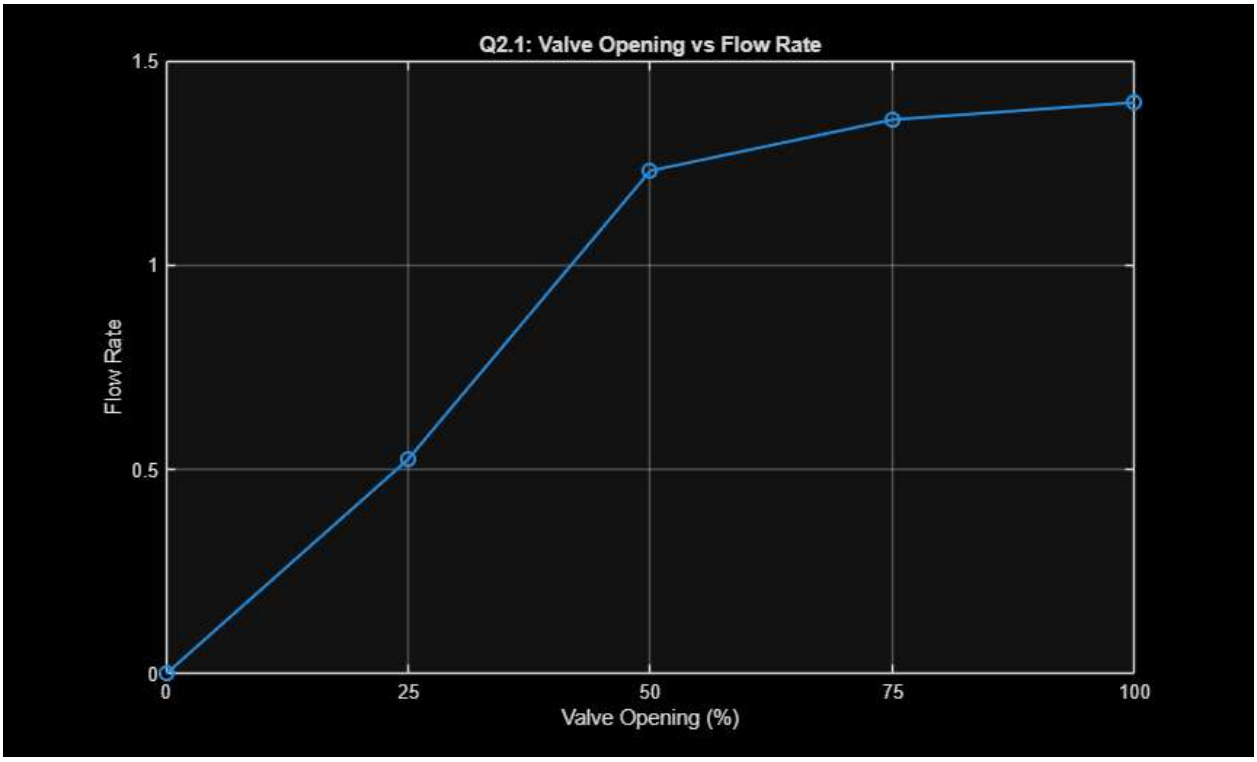
% Experimental data
opening = [0, 25, 50, 75, 100];        % Valve opening (%)
flow_rate = [0, 0.525, 1.231, 1.356, 1.398]; % Flow rate

% Plot
figure;
plot(opening, flow_rate, '-o', 'LineWidth', 1.5, 'MarkerSize', 7);

% Labels and title
xlabel('Valve Opening (%)');
ylabel('Flow Rate');
title('Q2.1: Valve Opening vs Flow Rate');

% Axis settings
xticks(opening);
xlim([0 100]);
ylim([0 1.5]);

% Grid
grid on;
box on;
```



Published with MATLAB® R2025a

ARDUINO CODES

1. FLOWMETER CALIBRATION
2. CODE FOR THE ENTIRE SET-UP (CASE-1 + CASE-2)

THE FOLLOWING CODE IS FOR THE CALIBRATION OF FLOWMETER

```
// Pump Testing Code

/*
  Pump 1: LPWM=32, RPWM=33, R_EN=26, L_EN=13
  Pump 2: LPWM=18, RPWM=19, R_EN=17, L_EN=16
  Pump 3: LPWM=23, RPWM=25, R_EN=22, L_EN=21

  Solenoid Valve MOSFET = GPIO 27
  Flowmeter             = GPIO 14
  Float Switch          = GPIO 35
  External Switch       = GPIO 34
*/

// Pump 1 pins
#define LPWM_1 32
#define RPWM_1 33
#define R_EN_1 26
#define L_EN_1 13

// Pump 2 pins
#define RPWM_2 19
#define LPWM_2 18
#define R_EN_2 17
#define L_EN_2 16
```

```
// Pump 3 pins
#define RPWM_3 25
#define LPWM_3 23
#define R_EN_3 22
#define L_EN_3 21

#define SOV 27
#define FLOWMETER 14

volatile uint32_t pulseCount = 0;

volatile unsigned long firstPulseTime = 0;
volatile unsigned long lastPulseTime = 0;
volatile bool firstPulseDetected = false;

const unsigned long SAMPLE_INTERVAL_MS = 250;
unsigned long lastSampleTime = 0;

// PWM configuration
const uint32_t PWM_FREQ = 10000;
const uint8_t PWM_RESOLUTION = 8;
const uint16_t PWM_MAX = 255;

// Flowmeter interrupt
void IRAM_ATTR pulseCounter()
{
  unsigned long pulseTime = millis();

  if (!firstPulseDetected)
  {
    firstPulseTime = pulseTime;
    firstPulseDetected = true;
  }

  lastPulseTime = pulseTime;
  pulseCount++;
}

void setup()
{
```

```
Serial.begin(115200);

// Configure enable pins
pinMode(R_EN_1, OUTPUT);
pinMode(L_EN_1, OUTPUT);
pinMode(R_EN_2, OUTPUT);
pinMode(L_EN_2, OUTPUT);
pinMode(R_EN_3, OUTPUT);
pinMode(L_EN_3, OUTPUT);
pinMode(SOV, OUTPUT);

// Enable all three BTS7960 drivers
digitalWrite(R_EN_1, HIGH);
digitalWrite(L_EN_1, HIGH);

digitalWrite(R_EN_2, HIGH);
digitalWrite(L_EN_2, HIGH);

digitalWrite(R_EN_3, HIGH);
digitalWrite(L_EN_3, HIGH);

// Solenoid ON
digitalWrite(SOV, HIGH);

// Configure PWM pins
ledcAttach(LPWM_1, PWM_FREQ, PWM_RESOLUTION);
ledcAttach(RPWM_1, PWM_FREQ, PWM_RESOLUTION);

ledcAttach(LPWM_2, PWM_FREQ, PWM_RESOLUTION);
ledcAttach(RPWM_2, PWM_FREQ, PWM_RESOLUTION);

ledcAttach(LPWM_3, PWM_FREQ, PWM_RESOLUTION);
ledcAttach(RPWM_3, PWM_FREQ, PWM_RESOLUTION);

// Pump 1: 50% duty cycle
ledcWrite(LPWM_1, 128);
ledcWrite(RPWM_1, 0);

// Pump 2: 100% duty cycle
ledcWrite(LPWM_2, 220);
```

```
  ledcWrite(RPWM_2, 0);

  // Pump 3: 50% duty cycle
  ledcWrite(LPWM_3, 128);
  ledcWrite(RPWM_3, 0);

  // Flowmeter pulse counting
  pinMode(FLOWMETER, INPUT_PULLUP);

  attachInterrupt(
    digitalPinToInterrupt(FLOWMETER),
    pulseCounter,
    FALLING
  );

  lastSampleTime = millis();

  // CSV header for Serial Monitor
  Serial.println(
    "Elapsed_ms,Pulse_Count,First_Pulse_ms,Last_Pulse_ms"
  );
}

void loop()
{
  unsigned long currentTime = millis();

  if (currentTime - lastSampleTime >= SAMPLE_INTERVAL_MS)
  {
    lastSampleTime = currentTime;

    uint32_t pulses;
    unsigned long firstTime;
    unsigned long lastTime;
    bool detected;

    // Safely copy ISR variables
    noInterrupts();

    pulses = pulseCount;
```

```
    firstTime = firstPulseTime;
    lastTime = lastPulseTime;
    detected = firstPulseDetected;

    interrupts();

    // Print CSV data to Serial Monitor
    Serial.print(currentTime);
    Serial.print(",");

    Serial.print(pulses);
    Serial.print(",");

    if (detected)
      Serial.print(firstTime);
    else
      Serial.print(-1);

    Serial.print(",");

    if (detected)
      Serial.println(lastTime);
    else
      Serial.println(-1);
  }
}
```

THE FOLLOWING CODE IS FOR THE FULL SET-UP (CLOSED LOOP SYSTEM + FLUSH), i.e., case-1 and case-2

```
/*
  ESP32 + BTS7960 Pump Control + Flowmeter (flow rate sent over WiFi)

  Pump 1: LPWM=32, RPWM=33, R_EN=26, L_EN=13
  Pump 2: LPWM=18, RPWM=19, R_EN=17, L_EN=16
  Pump 3: LPWM=23, RPWM=25, R_EN=22, L_EN=21
  SOV: GPIO 27
  Flowmeter: GPIO 14
  Float switch: GPIO 35 (LOW = triggered)
  ON/OFF switch: GPIO 34 (HIGH = OFF, LOW = ON)
  LED: GPIO 2

  Cycle: Case 1 (3 min) -> Case 2 flush/refill -> LED blink (5 s) -> Case
1 ...

  Flow rate is sent via HTTP GET to
http://<SERVER_IP>:5000/flow?value=<L/min>
  A separate FreeRTOS task (core 0) does the networking so the pump/state
  machine loop is never blocked by WiFi or HTTP delays.
*/

#include <Arduino.h>
#include <WiFi.h>
#include <HTTPClient.h>

// ---------------- WIFI SETTINGS (EDIT THESE) ----------------

// Enter your existing Wi-Fi network credentials
const char* WIFI_SSID     = "TP-Link_F0FC";
const char* WIFI_PASSWORD = "79131773";
const char* SERVER_IP     = "192.168.1.103";   // IP of the PC running the
Python script
const int   SERVER_PORT   = 5000;
```

```
const unsigned long WIFI_SEND_INTERVAL_MS = 500;

// ---------------- PIN DEFINITIONS ----------------

const int P1_LPWM = 32, P1_RPWM = 33;
const int P1_REN = 26, P1_LEN = 13;

const int P2_LPWM = 18, P2_RPWM = 19;
const int P2_REN = 17, P2_LEN = 16;

const int P3_LPWM = 23, P3_RPWM = 25;
const int P3_REN = 22, P3_LEN = 21;

const int SOLENOID_PIN = 27;
const int FLOW_PIN = 14;
const int FLOAT_PIN = 35;
const int MODE_PIN = 34;   // ON/OFF switch
const int LED_PIN = 2;

const int PWM_FREQ = 20000;
const int PWM_BITS = 8;

const int PUMP1_DUTY = 128;
const int PUMP2_DUTY = 255;
const int PUMP3_DUTY = 128;

const float CALIBRATION_FACTOR = 0.02078;
const float CALIBRATION_BIAS = 0.109172;

const unsigned long FLOW_INTERVAL_MS = 500;
const unsigned long ZERO_FLOW_TIMEOUT_MS = 1000;
const unsigned long ZERO_FLOW_SETTLE_MS = 1000;
const unsigned long LED_BLINK_MS = 500;
const unsigned long SWITCH_DEBOUNCE_MS = 50;

const unsigned long CASE1_DURATION_MS = 3UL * 60UL * 1000UL;  // 3 minutes
const unsigned long BLINK_DURATION_MS = 5000UL;               // 5 seconds

// ---------------- INTERRUPT FLAGS ----------------
```

```
volatile uint32_t pulseCount = 0;
uint32_t pulses = 0;

void IRAM_ATTR flowISR() {
  pulseCount++;
}

// ---------------- STATE MACHINE ----------------

enum OperatingMode {
  CASE1_NORMAL,
  CASE2_FLUSH
};

enum FlushState {
  FLUSH_WAIT_ZERO,
  FLUSH_ZERO_SETTLE,
  FLUSH_PUMPS_13,
  FLUSH_BLINK
};

OperatingMode mode = CASE1_NORMAL;
FlushState flushState = FLUSH_WAIT_ZERO;

bool systemOn = false;

// Switch debounce (polled)
bool rawModePin = HIGH;
bool stableModePin = HIGH;
unsigned long lastModeChangeMs = 0;

unsigned long stateStartMs = 0;
unsigned long lastFlowPulseMs = 0;
unsigned long lastFlowSampleMs = 0;
unsigned long lastLedToggleMs = 0;

bool ledState = false;

uint32_t totalPulses = 0;
```

```
volatile float flowRateLMin = 0.0f;   // read by the WiFi task (32-bit
float write is atomic on ESP32)
float totalLitres = 0.0f;

// ---------------- ACTUATOR FUNCTIONS ----------------

void stopPump1() {
  analogWrite(P1_LPWM, 0);
  analogWrite(P1_RPWM, 0);
  digitalWrite(P1_LEN, LOW);
  digitalWrite(P1_REN, LOW);
}

void stopPump2() {
  analogWrite(P2_LPWM, 0);
  analogWrite(P2_RPWM, 0);
  digitalWrite(P2_LEN, LOW);
  digitalWrite(P2_REN, LOW);
}

void stopPump3() {
  analogWrite(P3_LPWM, 0);
  analogWrite(P3_RPWM, 0);
  digitalWrite(P3_LEN, LOW);
  digitalWrite(P3_REN, LOW);
}

void runPump1() {
  digitalWrite(P1_REN, HIGH);
  digitalWrite(P1_LEN, HIGH);
  analogWrite(P1_RPWM, 0);
  analogWrite(P1_LPWM, PUMP1_DUTY);
}

void runPump2() {
  digitalWrite(P2_REN, HIGH);
  digitalWrite(P2_LEN, HIGH);
  analogWrite(P2_RPWM, 0);
  analogWrite(P2_LPWM, PUMP2_DUTY);
}
```

```
void runPump3() {
  digitalWrite(P3_REN, HIGH);
  digitalWrite(P3_LEN, HIGH);
  analogWrite(P3_RPWM, 0);
  analogWrite(P3_LPWM, PUMP3_DUTY);
}

void stopAllActuators() {
  stopPump1();
  stopPump2();
  stopPump3();
  digitalWrite(SOLENOID_PIN, LOW);
  digitalWrite(LED_PIN, LOW);
  ledState = false;
}

// ---------------- MODE TRANSITIONS ----------------

void enterCase1() {
  mode = CASE1_NORMAL;
  flushState = FLUSH_WAIT_ZERO;
  stateStartMs = millis();   // start 3-minute timer

  digitalWrite(LED_PIN, LOW);
  ledState = false;

  digitalWrite(SOLENOID_PIN, HIGH);
  runPump2();
  stopPump3();

  if (digitalRead(FLOAT_PIN) == HIGH) {
    runPump1();
  } else {
    stopPump1();
  }

  Serial.println("CASE 1: Normal operation (3 min)");
}
```

```
void enterCase2() {
  mode = CASE2_FLUSH;
  flushState = FLUSH_WAIT_ZERO;
  stateStartMs = millis();

  digitalWrite(SOLENOID_PIN, LOW);
  stopPump1();
  stopPump3();
  runPump2();

  digitalWrite(LED_PIN, LOW);
  ledState = false;

  Serial.println("CASE 2: Flush started");
}

void startSystem() {
  systemOn = true;
  Serial.println("SYSTEM ON");
  enterCase1();   // always restart from Case 1 with a fresh 3-min timer
}

void stopSystem() {
  systemOn = false;
  stopAllActuators();
  Serial.println("SYSTEM OFF");
}

// ---------------- ON/OFF SWITCH (HIGH = OFF, LOW = ON) ----------------

void processModeSwitch() {
  unsigned long now = millis();
  bool raw = digitalRead(MODE_PIN);

  if (raw != rawModePin) {
    rawModePin = raw;
    lastModeChangeMs = now;
  }
```

```
  if ((now - lastModeChangeMs) >= SWITCH_DEBOUNCE_MS && rawModePin !=
stableModePin) {
    stableModePin = rawModePin;

    // Toggle only on the press (HIGH -> LOW), not on release
    if (stableModePin == LOW) {
      if (systemOn) {
        stopSystem();
      } else {
        startSystem();
      }
    }
  }
}

// ---------------- FLOWMETER PROCESSING ----------------

void processFlowmeter() {
  unsigned long now = millis();

  noInterrupts();
  uint32_t newPulses = pulseCount;
  pulseCount = 0;
  interrupts();

  if (newPulses > 0) {
    pulses += newPulses;
    totalPulses += newPulses;
    lastFlowPulseMs = now;
  }

  if (now - lastFlowSampleMs >= FLOW_INTERVAL_MS) {
    unsigned long elapsed = now - lastFlowSampleMs;
    lastFlowSampleMs = now;

    float elapsedSeconds = (float)elapsed / 1000.0f;

    float rate = 0.0f;
    if (pulses > 0) {
```

```
      rate = ((float)pulses / elapsedSeconds) * CALIBRATION_FACTOR +
CALIBRATION_BIAS;
    }

    flowRateLMin = rate;
    totalLitres += rate * elapsedSeconds / 60.0f;
    pulses = 0;

    if (systemOn) {
      Serial.print("Flow rate: ");
      Serial.print(rate, 3);
      Serial.println(" L/min");
    }
  }
}

bool isZeroFlow() {
  return (millis() - lastFlowPulseMs) >= ZERO_FLOW_TIMEOUT_MS;
}

// ---------------- WIFI TASK (runs on core 0) ----------------

void wifiTask(void* parameter) {
  char url[96];

  for (;;) {
    if (WiFi.status() != WL_CONNECTED) {
      // Auto-reconnect is enabled; just wait and let the stack retry.
      vTaskDelay(pdMS_TO_TICKS(500));
      continue;
    }

    snprintf(url, sizeof(url), "http://%s:%d/flow?value=%.3f",
             SERVER_IP, SERVER_PORT, (float)flowRateLMin);

    HTTPClient http;
    http.setConnectTimeout(1000);
    http.setTimeout(1000);

    if (http.begin(url)) {
```

```
      http.GET();   // response code ignored; next send retries anyway
      http.end();
    }

    vTaskDelay(pdMS_TO_TICKS(WIFI_SEND_INTERVAL_MS));
  }
}

// ---------------- CASE 1 ----------------

void runCase1() {
  digitalWrite(SOLENOID_PIN, HIGH);
  runPump2();
  stopPump3();

  if (digitalRead(FLOAT_PIN) == LOW) {
    stopPump1();
  } else {
    runPump1();
  }

  if (millis() - stateStartMs >= CASE1_DURATION_MS) {
    enterCase2();
  }
}

// ---------------- CASE 2 FLUSH SEQUENCE ----------------

void runFlush() {
  unsigned long now = millis();
  bool floatTriggered = (digitalRead(FLOAT_PIN) == LOW);

  switch (flushState) {

    case FLUSH_WAIT_ZERO:
      stopPump1();
      stopPump3();
      runPump2();

      if (isZeroFlow()) {
```

```
        flushState = FLUSH_ZERO_SETTLE;
        stateStartMs = now;
        Serial.println("Zero flow detected; waiting 1 second");
      }
      break;

    case FLUSH_ZERO_SETTLE:
      runPump2();

      if (now - stateStartMs >= ZERO_FLOW_SETTLE_MS) {
        stopPump2();
        runPump1();
        runPump3();

        flushState = FLUSH_PUMPS_13;
        Serial.println("Pump 1 + Pump 3 ON; Pump 2 OFF");
      }
      break;

    case FLUSH_PUMPS_13:
      stopPump2();
      runPump1();
      runPump3();

      if (floatTriggered) {
        flushState = FLUSH_BLINK;
        stateStartMs = now;
        lastLedToggleMs = now;
        ledState = true;
        digitalWrite(LED_PIN, HIGH);

        Serial.println("Float LOW: manually turn OFF the ball valve (LED
blinking 5 s)");
      }
      break;

    case FLUSH_BLINK:
      // Pumps 1 + 3 stay on while the LED blinks for 5 seconds.
      stopPump2();
      runPump1();
```

```
      runPump3();

      if (now - lastLedToggleMs >= LED_BLINK_MS) {
        lastLedToggleMs = now;
        ledState = !ledState;
        digitalWrite(LED_PIN, ledState);
      }

      if (now - stateStartMs >= BLINK_DURATION_MS) {
        digitalWrite(LED_PIN, LOW);
        ledState = false;
        Serial.println("Blink done; returning to Case 1");
        enterCase1();
      }
      break;
  }
}

// ---------------- SETUP ----------------

void setup() {
  Serial.begin(115200);

  const int outputPins[] = {
    P1_LPWM, P1_RPWM, P1_REN, P1_LEN,
    P2_LPWM, P2_RPWM, P2_REN, P2_LEN,
    P3_LPWM, P3_RPWM, P3_REN, P3_LEN,
    SOLENOID_PIN, LED_PIN
  };

  for (int pin : outputPins) {
    pinMode(pin, OUTPUT);
    digitalWrite(pin, LOW);
  }

  // GPIO34/35 have no internal pull resistors -> need external pull-ups.
  pinMode(MODE_PIN, INPUT);
  pinMode(FLOAT_PIN, INPUT);
  pinMode(FLOW_PIN, INPUT_PULLUP);
```

```
  analogWriteResolution(P1_LPWM, PWM_BITS);
  analogWriteResolution(P2_LPWM, PWM_BITS);
  analogWriteResolution(P3_LPWM, PWM_BITS);

  analogWriteFrequency(P1_LPWM, PWM_FREQ);
  analogWriteFrequency(P2_LPWM, PWM_FREQ);
  analogWriteFrequency(P3_LPWM, PWM_FREQ);

  attachInterrupt(digitalPinToInterrupt(FLOW_PIN), flowISR, FALLING);

  rawModePin = digitalRead(MODE_PIN);
  stableModePin = rawModePin;    // a button held at boot won't trigger a
toggle
  lastModeChangeMs = millis();

  lastFlowPulseMs = millis();
  lastFlowSampleMs = millis();

  stopSystem();       // Initially everything is off

  // WiFi: non-blocking start, so pump control works even if WiFi is down.
  WiFi.mode(WIFI_STA);
  WiFi.setAutoReconnect(true);
  WiFi.begin(WIFI_SSID, WIFI_PASSWORD);

  xTaskCreatePinnedToCore(wifiTask, "wifiTask", 8192, NULL, 1, NULL, 0);

  Serial.println("Controller initialized.");
}

// ---------------- MAIN LOOP ----------------

void loop() {
  processFlowmeter();
  processModeSwitch();

  if (!systemOn) return;

  if (mode == CASE1_NORMAL) {
    runCase1();
```

```
  } else {
    runFlush();
  }
}
```

code with wifi